



AULA 4
UNIDADE 5

Branches



Branches

- *Branches* são visões de uma mesma base de código, que podem evoluir de forma independente.
- Assim como os galhos de uma árvore, todos os *branches* tem uma mesma raiz da qual ramificações são criadas:
 - No GitHub, a raiz é a *master* ou *main*.
 - O *branch* inicial é criado pelo comando *git init*.
- A razão da existência de *branches* é permitir que desenvolvedores tenham liberdade em desenvolver uma nova funcionalidade sem a comunicação constante com o resto da equipe.
- Os *branches* podem ser unificados (*merge*), sendo este o momento para a equipe decidir quais alterações são mantidas no *branch* raiz.



Branches

- Continuando o exemplo da aula passada, considere que *usuario1* deseja implementar uma nova funcionalidade do sistema.
- Seja esta nova funcionalidade uma função *add(a, b)*. Seria um exemplo simples apenas de ilustração. Como ainda não tem certeza que será vantajosa para o projeto como um todo, *usuario1* cria um *branch*:

```
$ git branch calculator
```
- Com o novo *branch* criado, o *usuario1* pode então alterar o *branch* corrente:

```
$ git checkout calculator
```
- A qualquer momento, o comando *git branch* lista os *branches* disponíveis e aponta o *branch* corrente.
- No momento do *checkout*, todos os arquivos do *branch* pai estão disponíveis no recém criado.



Branches

- O *usuario1* só pode alterar para um novo *branch* se as modificações feitas no antigo estiverem salvas.

- No novo *branch*, o usuário cria o arquivo *src/calculator.py* com o seguinte conteúdo:

```
def add(a, b):  
    return a + b
```

- Novamente, o *usuario1* precisa adicionar e submeter:

```
$ git add src/calculator.py
```

```
$ git commit -m "Add function"
```

- Observe que até agora o *branch* é local.



Branches

- Com a criação do *branch*, *calculator* e *main* representam visões diferentes da árvore de código. Para integrá-las, é preciso fazer um *merge*:

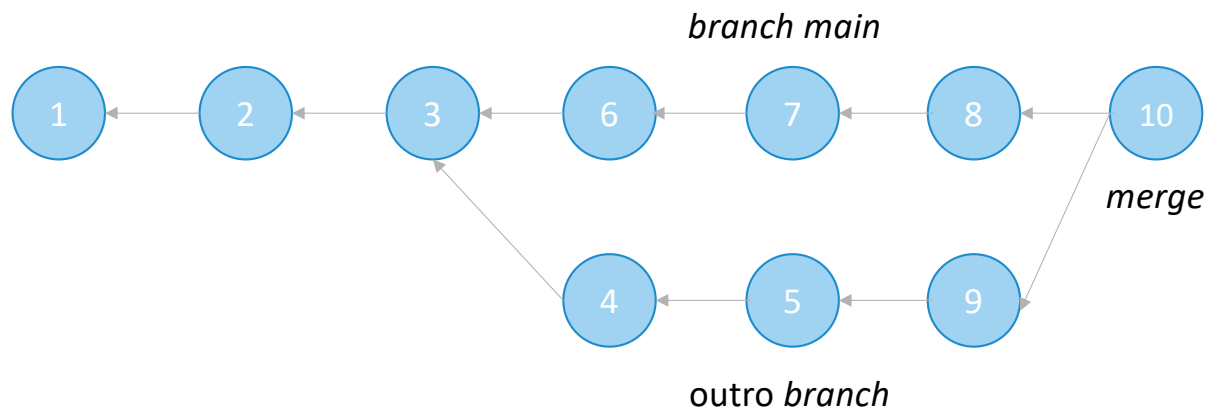
```
$ git checkout main  
$ git merge calculator
```

- Se houver discrepâncias entre os *branches*, haverá um conflito.
- Como mostrado na aula passada, o Git irá gerar um arquivo com marcadores indicando a diferença.
- Cabe ao desenvolvedor que iniciou o *merge* resolver os conflitos e submeter a versão final do arquivo.



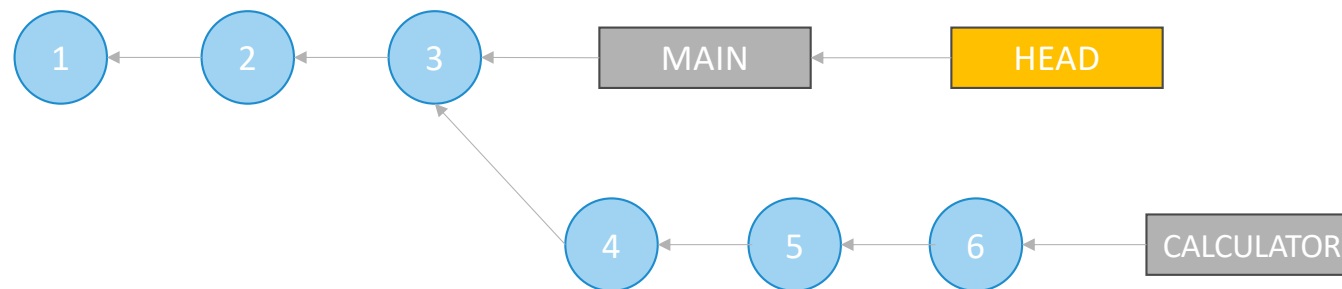
Grafos de *Commits*

- *Commits* possuem vários antecessores.



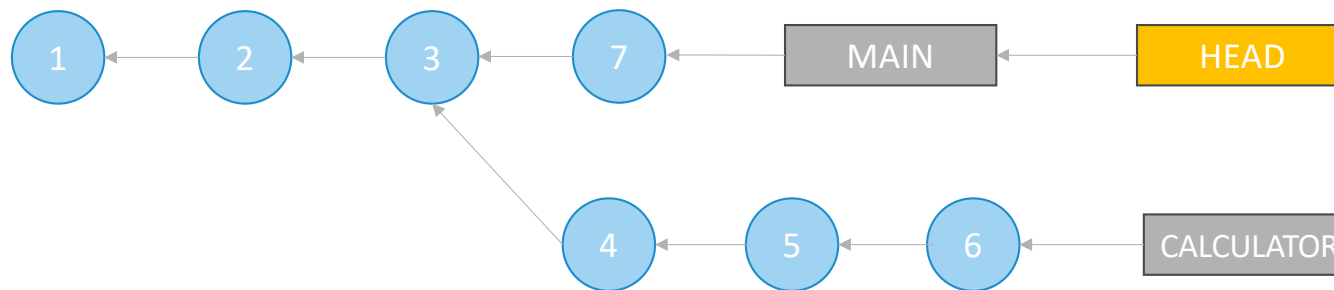
Grafos de *Commits*

- Para cada *branch*, o Git mantém uma variável que aponta para o último *commit* feito.
- A variável HEAD contém o nome da variável que armazena o identificador do último *commit* do *branch* corrente.



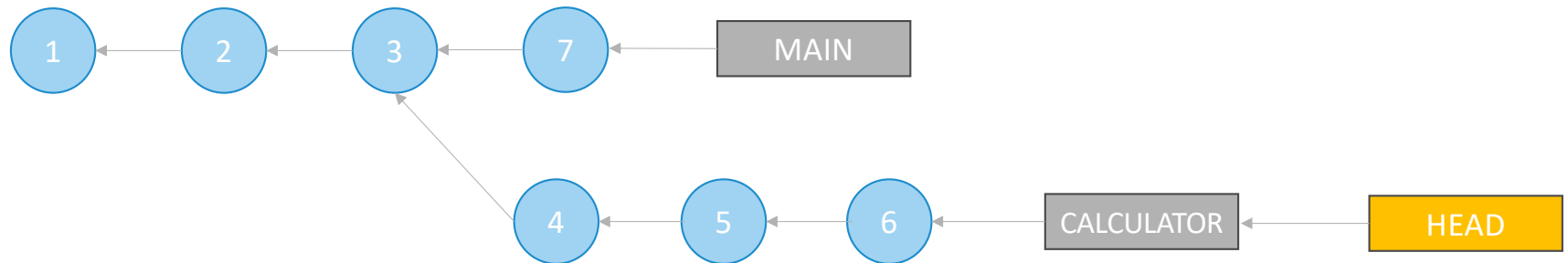
Grafos de *Commits*

```
$ git commit -m "Alterações no main."
```



Grafos de *Commits*

\$ git checkout calculator



Branches Remotos

- O *usuario1* criou o *branch calculator* localmente.
- Para submetê-lo ao servidor remoto:

```
$ git push -u origin calculator
```
- O parâmetro *-u* só precisa ser usado na primeira vez, indica que vamos querer sincronizar através de *git pull*.
- O *usuario2* deve então recuperar o *branch* para seu repositório local:

```
$ git pull
```

```
$ git checkout -t origin/calculator
```
- O *-t* significa *tracking*, ou seja, o *branch* local do *usuario2* irá rastrear o remoto.



Pull Request

- O desenvolvedor cria um *branch* e faz as alterações.
- Entretanto, não faz o *merge* no *branch main*, mas sim submete para a avaliação do dono do repositório (ou outro usuário com privilégios administrativos).
- O dono do repositório avalia as alterações:
 - O revisor faz um *pull* do *branch* proposto no seu repositório local e faz um *merge*.
 - O GitHub fornece uma interface *web* no formato de fórum na qual alterações podem ser discutidas antes do *merge*.



Squash

- Apesar de termos afirmado que o ideal são *commits* pequenos, para revisão de *pull requests* é boa prática consolidar vários *commits* em um só.
- Unir os cinco últimos *commits* do *branch* corrente:
 - `$ git rebase -i HEAD~5`
- Um editor de texto será aberto, de forma semelhante a um *git commit* sem o parâmetro *-m*.
- O desenvolvedor deve substituir *pick* por *squash* em todas as linhas, exceto a primeira.
- Uma vez que o arquivo for salvo, o *squash* estará finalizado.



Forks

- É uma funcionalidade específica do GitHub para clonar repositórios.
- Entretanto, não é uma cópia local.
- O repositório alvo do *fork* é copiado no próprio GitHub.
- Um desenvolvedor pode fazer alterações no novo repositório e submeter um *pull request* para os mantenedores do repositório original.
- Apesar do funcionamento semelhante a um *branch*, o *fork* permite contribuições em um repositório público de desenvolvedores que não estão cadastrados entre os colaboradores do repositório.



Conclusão

- Controle de versão de código é essencial ao Fluxo de Valor Tecnológico.
- Git é o sistema de controle de versão mais utilizado na atualidade.
- Nesta aula, vimos como através de *branches*, *pull requests* e *forks*, desenvolvedores conseguem cooperar em um projeto de *software*.
- O GitHub é um serviço construído tendo por base o protocolo Git, mas adiciona várias funcionalidades que auxiliam na cooperação.





GRADUAÇÃO ONLINE **FGV**

TECNOLÓGICA